# DA6400 : Reinforcement Learning (Jan-May 2026)

Programming Assignment 2

Deadline: April 2, 2026 11:59 PM

## 1 Instructions

1. **Start early!**
2. Write complete Python code for this assignment. Include your `requirements.txt` and/or conda yaml file along with a README to reproduce your experiments.
3. Submit a report containing the experimental results and answers to the questions below. The answers must be clear, concise, and to the point. A single line explanation is sufficient if it serves the purpose. Unnecessary verbose can be penalized. The report has a strict limit of 10 pages.
4. The maximum marks without the bonus question is 90. The marks distribution is provided below. 90M $\equiv$ 90 marks.
5. We expect one submission per group of 3 members. This assignment is designed for team effort. Each member must contribute for successful completion of the assignment.
6. Read the full PDF before starting any experiments. Take a note of all the logs to store. Advice: Run for 2 seeds to ensure everything is okay. Then run all.
7. It is advised to use a code editor (e.g., VSCode) to handle the code and experiments.
8. **Please strictly follow the academic code of conduct. Plagiarism will be penalized.**

## 2 Environment

For this programming task, you should utilize the following Gymnasium environment for training and evaluating your policies. The associated link contains the environment description. Please use the exact version of the environment as specified:

MountainCar-v0: The Mountain Car (MC) MDP (Figure 1) is a deterministic MDP that consists of a car placed stochastically at the bottom of a sinusoidal valley. The only possible actions being the accelerations that can be applied to the car in either direction or no acceleration at all. The goal of the MDP is to strategically accelerate the car to reach the goal state on top of the right hill. There are two versions of the mountain car domain in gymnasium: one with discrete actions and one with continuous. You should use the one

with discrete actions. Check the environment documentation for further details.

The observation space is continuous, which can be handled by the algorithms you use for this assignment. Hence, unlike PA1, you should **NOT** use any type of binning of the observation space. Important pointers:

1. Use $\gamma = 0.99$.
2. The default MountainCar-v0 implementation ends an episode if the car reaches the target (termination) or if the episode length exceeds 200 timesteps (truncation). **You should modify the truncation length to 2000 timesteps for your experiments.**
3. Do **NOT** use any reward shaping. Use the default reward structure in the MountainCar-v0 documentation.

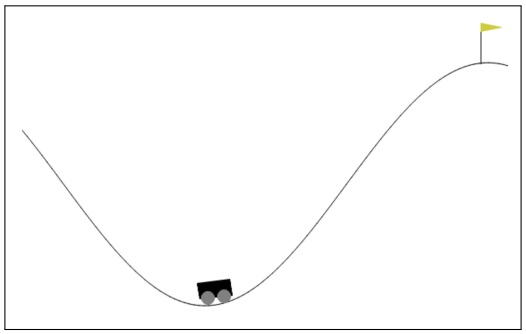


Figure 1: Mountain Car environment

# 3 Deep Q-Networks on Mountain Car

1. Write Python code for the vanilla DQN algorithm and run it on MountainCar-v0. You should use the most basic version of DQN with $\epsilon$-greedy exploration (apply $\epsilon$-decaying if necessary), fixed replay buffer size, hard target network updates, uniformly random sampling for experience replay, and Adam optimizer. The choice of hyperparameters is left to you. But make sure to choose the optimal hyperparameters for the Mountain Car setting. Further, you should use a neural network representation (for Q-function) that is neither overparameterized nor underparameterized[1]. Use Kaiming initialization for the (ReLU-based) Q-network.

   Do not use DQN variants such as double Q-networks, recurrent Q-networks, dueling networks, distributional RL, multi-step DQN, Rainbow DQN, soft target network updates, etc. Use the vanilla DQN in the original DQN paper. **[5M]**

[1] If you use an unnecessarily large capacity neural network, the experiments will be expensive and time-consuming. On the other hand, an insufficient capacity may cause a performance bottleneck.

2. Run the experiment for sufficient duration so as to achieve a near-optimal policy. Reaching near-optimal performance is mandatory for attaining the respective marks. **[8M]**

   **Note:** Unless specified otherwise, run all experiments in this assignment for 15 random seeds/runs. A random seed corresponds to the Q-network initialization and the initial start state of the agent.

   (a) Plot return vs. timesteps/episodes curves for vanilla DQN on Mountain Car (Figure 2). Plot the mean performance along with confidence intervals. **[3M]**

   (b) Give a concise description of the results with interesting comments, if found any. Explain and reason out the optimal behavior of the car. Does the car directly accelerate towards the target? **Tip:** for intuition, it is good to render the environment and visualize the car attempting to solve the task. **[5M]**

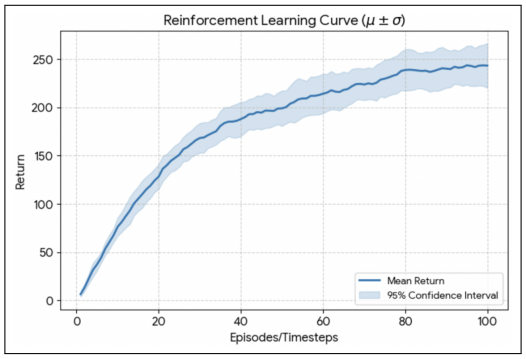



Figure 2: Sample plot for (average) return vs episodes/timesteps for an algorithm in an environment. This plot is just for illustration.

3. **Length of the episode:** We modified the (artificial) episode truncation length to 2000 timesteps. This effectively means that if the agent does not reach the terminal state by the end of 2000 timesteps, we truncate the episode and start a new one. **[12M]**

   (a) Run DQN with the truncation length of 200 timesteps (gymnasium default) and 1000 timesteps and plot the results besides the one in Question 2. **[2M]**

   (b) Compare the three scenarios. Is there any change if the truncation length is increased/decreased? **[2M]**

   (c) The original Mountain Car (Andrew Moore; Sutton & Barto) had no artificial episode truncation. Why do you think then these were introduced in Gymnasium? **[3M]**

   (d) How can introducing truncation be harmful? What should be an (approximate) appropriate truncation length so as to evaluate RL algorithms fairly on Mountain

Car? Is it $200, 1000, 2000$, or something else? Justify. **[5M]**

4. Modify the code to implement the following DQN variant (keep truncation length as 2000 for all following experiments): the vanilla implementation involves the agent sampling a mini-batch of transitions (from the replay buffer) once per timestep to update the Q-network. You should put a loop around this by sampling multiple batches and updating the Q-network **sequentially**. The number of times the update is done per timestep is referred to as the **replay factor ($\rho$)**. Hence, a replay factor of $\rho$ implies sampling a mini-batch and using it to update the network parameters sequentially $\rho$ times per timestep. Note that $\rho = 1$ corresponds to the vanilla DQN. **[65M]**

   (a) **Run experiments for DQN with $\rho = \{1, 2, 4, 8\}$. [18M]**

      i. Compare the results. As usual, use the mean performance over multiple seeds for each $\rho$. Follow the template of the illustrative plots in Figures 3a and 3b for comparing the different DQN $\rho$-variants. **[3M]**

      ii. Comment if there are any differences between DQN with different $\rho$ values. Are they statistically significant? **[2M]**

      iii. Why or why not do the differences exist? Explain the effect of increasing the replay factor. Comment on the final policy performance and learning/sample efficiency (how fast the agent learns). **[5M]**

      iv. When and why is increasing $\rho$ advantageous and disadvantageous during deployment (practical application)? Support this answer using a (succinct) real-world research and development scenario. **[4M]**

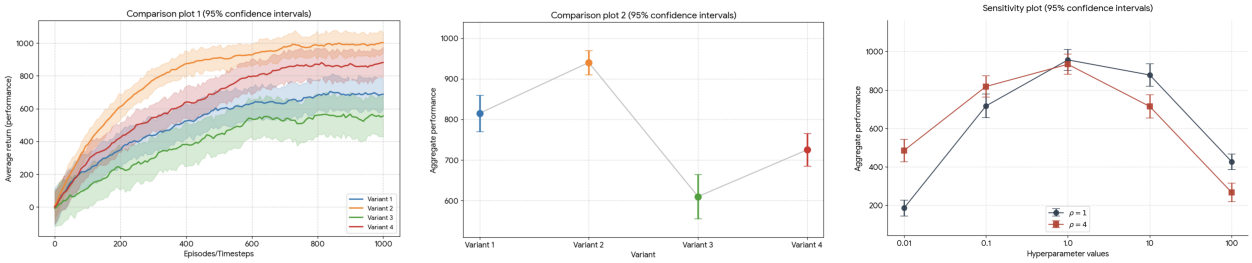
      v. In reinforcement learning, **planning** is defined as any computational process that uses a model (transition probabilities and rewards) of the environment to create or improve predictions/policies. How does DQN with $\rho > 1$ relate to this definition of planning? **[4M]**

   Important: The only thing changing between the above experiments is $\rho$. Rest all hyperparameters should remain the same for a fair comparison.

   (b) **Distribution of performance:** This corresponds to frequency distributions (smoothened if required) representing the aggregate performance of all the runs. One sample of the distribution should correspond to the aggregate performance of a single run. **[10M]**

      i. Plot the distributions for different replay factors and compare them. Are the distributions unimodal or multimodal? Are they skewed? Do they look like a Normal distribution? What do the distributions intuitively convey? **[3M]**

      ii. Are there any nuanced inferences (over what is already inferred in part a)




(a) Comparison plot 1 (b) Comparison plot 2 (c) Sensitivity plot

Figure 3: Sample plots for illustration. Aggregate performance in Figures 3b and 3c can be obtained using area under the curve (scaled down by the number of measurements). This can then be averaged over the number of runs and plotted with **95**% confidence intervals.

you can make with the generated performance distributions? If yes, explain. **[3M]**

iii. Can you think of scenarios (beyond the current DQN-MC setting) when visualizing the distribution of performance provides deeper insights into the behavior of algorithms that the usual learning curves? Explain. **[4M]**

(c) **Variability in performance (tolerance intervals):** Tolerance intervals capture the variability in the performance. Read more about them online. **[12M]**

i. Plot the mean performance with shaded regions representing $(\alpha = 0.05, \beta = 0.9)$-tolerance intervals. Take the logs from part (a) and plot tolerance intervals for different $\rho$ values. **[2M]**

ii. Compare the tolerance intervals of different $\rho$ values. What different information do these results provide over what you wrote in part (a)? **[3M]**

iii. Based on these results, provide insights into the reliability and robustness of DQN with different $\rho$-values. What can you say about the worst-case performance of the agent? **[4M]**

iv. What is the fundamental difference between tolerance and confidence intervals? When is either useful and/or misleading? If the number of runs tends to infinity, what do these intervals converge to? **[3M]**

(d) **Sensitivity analysis:** Take DQN with $\rho = \{1, 4\}$. Take two hyperparameters: mini-batch size and target network refresh rate. For each of these two hyperparameters, take two values smaller and two values larger than the value used in previous experiments[2]. Plot the sensitivity curves for these two hyperparameters. An illustration is provided in Figure 3c. **[25M]**

[2]Implicit assumption is that you used the optimal hyperparameter value for the previous experiments.

i. **Mini-batch size:** How sensitive is DQN ($\rho = 4$) compared to DQN ($\rho = 1$) for the mini-batch size? The total samples used at each training step is given by ($\rho *$ batch size). Given that the total samples is fixed, is it wiser to use a smaller $\rho$ with a larger batch size or a larger $\rho$ with a smaller batch size? Explain using your results. **[10M]**

ii. **Target network refresh rate:** Compare the sensitivities of DQN ($\rho = 1$) and DQN ($\rho = 4$) for the target network refresh rate. What happens when the refresh rate is too low or too high? Does $\rho > 1$ seem to learn more stably compared to $\rho = 1$ when the target network is updated more frequently? Explain either ways. **[10M]**

iii. Revisit your real-world scenario discussed in Question 4 (part a). Discuss it in context of the hyperparameter sensitivity experiments. Write if here there are any lessons about deployment and robustness/reliability of DQN. **[5M]**

5. **Bonus:** It is conjectured that the way transitions are sampled from the replay buffer may affect the replay factor ($\rho$) in some manner. In all previous experiments, you used uniform sampling which samples each transition with equal probability. Perform experiments (15 random seeds) with Prioritized Experience Replay (PER) and check if prioritizing certain transitions over others affects increasing the replay factor in a positive/negative manner. PER prioritizes samples having a higher TD error, intuitively quantifying how "surprised" the agent was to see that sample. Does using PER diminish/elevate the effect of increasing the replay factor? Why? **[15M]**